

Linux 内核漏洞复现与分析实验报告

基于 KernJC 自动化环境构建工具的 CVE-2021-26708 复现

实验人: kungao

摘要

本实验基于 KernJC 自动化漏洞环境构建工具 [1], 对 Linux 内核漏洞 CVE-2021-26708 (VSOCK 竞争条件 Use-After-Free) 进行了完整的环境构建、PoC 适配与 KASAN 触发分析。实验记录了内核配置缺失、模块加载失败、QEMU 设备不兼容等典型问题的诊断和解决过程, 最终成功获取完整的 KASAN 报告, 验证了漏洞在 5.10.12 内核上的存在。

目录

1 实验环境	2
2 实验流程	2
2.1 第一阶段: KernJC 自动化内核编译	2
2.2 第二阶段: 内核配置修复	2
2.3 第三阶段: QEMU 虚拟设备添加	3
2.4 第四阶段: PoC 编译与执行	3
2.5 第五阶段: KASAN 报告获取	3
3 KASAN 分析报告	3
4 其他 CVE 尝试与失败分析	4
4.1 CVE-2016-10150 (KVM Use-After-Free, Kernel 4.8.12)	4
4.2 CVE-2021-22555 (Netfilter Heap OOB, Kernel 5.11.22)	5
4.3 CVE-2017-11176 (Netlink Use-After-Free, Kernel 4.12.1)	5
4.4 经验总结	5
5 遇到的问题与解决方案	5
5.1 QEMU 启动失败: write lock	5
5.2 内核模块无法加载	6
5.3 glibc 版本不兼容	6
5.4 AF_VSOCK 设备不可用	6
5.5 KernJC 自动化环境的局限性	6
6 结论	6

1 实验环境

操作系统	Ubuntu 22.04
QEMU 版本	6.2.0
内核编译工具链	gcc-9.5.0 (项目本地 toolchain)
KernJC 版本	0.8.3
目标漏洞	CVE-2021-26708
目标内核	5.10.12
漏洞类型	Use-After-Free (竞争条件)
受影响组件	net/vmw_vsock/af_vsock.c

2 实验流程

2.1 第一阶段: KernJC 自动化内核编译

```
1 cd /home/kun/Desktop/KernJC
2 source venv/bin/activate
3
4 # 查询漏洞信息和受影响版本
5 ./kjc query CVE-2021-26708
6
7 # 自动化编译内核
8 ./kjc build CVE-2021-26708
```

KernJC 自动完成:

1. 版本识别: 从候选版本 {5.11.11, 5.11.12} 中选取 5.10.12
2. 补丁补丁: 应用 gcc 兼容性补丁 (fix-sharp-escaping, fix-selinux 等)
3. Kconfig 图解析: 生成必要的内核配置
4. 内核编译: 生成 bzImage 和 rootfs.qcow2

2.2 第二阶段: 内核配置修复

首次编译后的内核缺少 CONFIG_VSOCKETS 和 CONFIG_USERFAULTFD, 导致 PoC 无法运行:

```
1 # 错误信息
2 socket: Address family not supported by protocol
3
4 # 检查配置
5 grep CONFIG_VSOCKETS .config
6 # 输出: # CONFIG_VSOCKETS is not set
7 grep CONFIG_USERFAULTFD .config
8 # 输出: # CONFIG_USERFAULTFD is not set
```

解决方案: 使用 `scripts/config` 增量启用缺失项后重建:

```
1 cd kernel/
2 scripts/config --enable CONFIG_VSOCKETS
3 scripts/config --enable CONFIG_VIRTIO_VSOCKETS
4 scripts/config --enable CONFIG_USERFAULTFD
5 make olddefconfig
6 make -j$(nproc)
7 # 输出: Kernel: arch/x86/boot/bzImage is ready (#2)
```

2.3 第三阶段: QEMU 虚拟设备添加

PoC 使用 `AF_VSOCK` 协议族, 需要 QEMU 提供 `virtio-vsock` 设备:

```
1 # 错误信息
2 connector connect: No such device
3
4 # 解决方案: 修改 start.sh 添加 vsock 设备
5 -device vhost-vsock-pci,guest-cid=3 \
6
7 # Host 侧加载模块
8 sudo modprobe vhost_vsock
```

2.4 第四阶段: PoC 编译与执行

```
1 # VM glibc 版本 (2.31) 低于宿主 (2.35), 必须 -static 编译
2 gcc -o /tmp/poc_26708 -pthread -static poc.c
3 ./kjc cp /tmp/poc_26708 85fd71034f66c0c0:/home/user/
4 ./kjc exec 85fd71034f66c0c0 /home/user/poc_26708
```

2.5 第五阶段: KASAN 报告获取

PoC 执行后通过 SSH 获取内核日志:

```
1 ./kjc attach 85fd71034f66c0c0
2 # VM 内:
3 dmesg | grep -B 2 -A 20 "BUG: KASAN"
```

3 KASAN 分析报告

以下为成功触发后的 KASAN `use-after-free` 完整报告:

```
1 BUG: KASAN: use-after-free in virtio_transport_notify_buffer_size+0x192/0x1c0
2 Read of size 4 at addr ffff88800632e828 by task poc_26708/215
3
4 Call Trace:
5 dump_stack+0x9a/0xccc
```

```

6  print_address_description.constprop.0+0x1c/0x220
7  ? virtio_transport_notify_buffer_size+0x192/0x1c0
8  kasan_report.cold+0x37/0x7c
9  ? virtio_transport_notify_buffer_size+0x192/0x1c0
10 virtio_transport_notify_buffer_size+0x192/0x1c0
11 vsock_update_buffer_size+0x15d/0x1f0
12 vsock_stream_setsockopt+0x6a6/0x820
13 __sys_setsockopt+0x158/0x270
14 __x64_sys_setsockopt+0xb9/0x150
15 do_syscall_64+0x33/0x40
16 entry_SYSCALL_64_after_hwframe+0x44/0xa9
17
18 Allocated by task 215:
19  kasan_save_stack+0x1b/0x40
20  __kasan_kmalloc.constprop.0+0xc2/0xd0
21  vsock_assign_transport+0x174/0x480
22  vsock_stream_connect+0x291/0xbd0
23  ...
24
25 Freed by task 215:
26  kfree+0xd8/0x3d0
27  vsock_remove_sock+0x2e0/0x2e0
28  vsock_stream_connect+0x767/0xbd0
29  ...

```

漏洞根因分析:位于 `virtio_transport_notify_buffer_size` 函数中,VSOCK socket 在调用 `vsock_remove_sock` 释放后被 `vsock_update_buffer_size` 的 callback 链路再次访问,形成 Use-After-Free。

4 其他 CVE 尝试与失败分析

在确定 CVE-2021-26708 为最终实验对象之前,本实验还尝试了以下两个漏洞,但因各种原因未能成功复现,其失败原因具有一定的代表性,特此记录。

4.1 CVE-2016-10150 (KVM Use-After-Free, Kernel 4.8.12)

失败原因: 内核版本与 QEMU CPU 模型不兼容。

Kernel 4.8.12 编译时使用了 `CONFIG_MATOM=y` (Intel Atom 指令集优化),而 QEMU 的 `qemu64` CPU 模型不包含 Atom 扩展指令,导致内核在启动阶段立即触发非法指令异常,陷入反复重启 (boot loop)。当尝试使用 `-cpu host` 并启用 KVM 加速时,现代 CPU 透传的指令集特性进一步加剧了与旧内核的兼容性问题,内核在 `sock_ioctl` 等处产生 General Protection Fault。

总结: 2016 年的内核版本 (4.8.x) 与现代 x86_64 CPU 微架构存在根本性的指令集兼容断层,即便通过 KernJC 编译通过,也无法在 QEMU/KVM 环境中正常启动。此类漏洞需要匹配特定年代的硬件或 QEMU 版本方可复现。

4.2 CVE-2021-22555 (Netfilter Heap OOB, Kernel 5.11.22)

失败原因: PoC 对内核模块配置要求过于苛刻。

该 PoC 需要 `ip6t_icmp6` 内核模块支持 IPv6 ICMP 匹配。虽然 KernJC 编译的内核中包含该模块的 `.ko` 文件, 但 Debian 最小化 rootfs 镜像缺少 `kmod` 包 (无 `modprobe/insmod` 命令), 无法在运行时加载模块。经 APT 安装 `kmod` 后, 又发现模块文件已被 `make clean` 清除, 需要重新增量编译模块。即便将模块文件通过 SCP 传入 VM, 仍面临: (1) 模块依赖链复杂; (2) 32 位 `compat` 路径对内核配置敏感; (3) PoC 本身对 `netfilter` 规则创建时序有严格要求的三点苛刻条件。

总结: 部分 PoC 对特定内核模块的运行时可用性要求极高, KernJC 默认生成的 rootfs 镜像在设计上未考虑模块动态加载场景, 导致此类 PoC 的复现成本远超预期。

4.3 CVE-2017-11176 (Netlink Use-After-Free, Kernel 4.12.1)

失败原因: 内核无法在 QEMU TCG 软件模拟模式下启动。

Kernel 4.12.1 与 4.8.12 类似, 在 `qemu64` CPU 模型下内核解压阶段即卡死 (Decompressing Linux... 后无任何输出)。由于该内核同样过于古老, `-cpu host + KVM` 透传方案不可用, 而 TCG 模拟又不兼容。

4.4 经验总结

三个失败案例揭示了 KernJC 工具在老旧内核 (4.x 系列) 复现场景中的系统性缺陷:

1. **CPU 兼容性:** 2016-2017 年内核与现代 CPU 微架构存在指令集断层, QEMU 的 CPU 模型选择范围 (`host` vs `qemu64`) 无法同时满足兼容性与性能需求。
2. **模块管理:** rootfs 镜像缺少运行时模块加载能力, 导致依赖 `.ko` 文件的 PoC 无法执行。
3. **配置完备性:** Kconfig 图的依赖解析不完整, 未自动识别某些漏洞所需的非直观内核配置项。

这些失败经验反过来验证了选择 CVE-2021-26708 的正确性: 该漏洞基于内核 5.10.12 (相对现代), 所需配置项可控 (`VSOCKETS + USERFAULTFD`), QEMU 设备支持成熟 (`virtio-vsock-pci`), PoC 使用标准系统调用而不依赖复杂的模块加载链。

5 遇到的问题与解决方案

5.1 QEMU 启动失败: write lock

现象: Failed to get "write" lock 错误导致 QEMU 无法启动。

诊断: 使用 `lsof rootfs.qcow2` 查找持有锁的进程, `pkill -9 qemu` 清理残留进程。如锁仍然存在, 复制一份新的 rootfs 镜像并使用新文件启动。

5.2 内核模块无法加载

现象: VM 内 `modprobe` 和 `insmod` 命令缺失, 无法加载 `ip6t_icmp6` 等内核模块。

根因: KernJC 的 `rootfs` 基于 Debian 最小化镜像, 未安装 `kmod` 包 (`create-image.sh` 的 `PREINSTALL_PKGS` 未包含 `kmod`)。

解决方案: VM 内 `apt-get install kmod`, 或改用内核内置 (`=y`) 方式重编译。本次实验对 CVE-2021-26708 采用后者 (增量编译内核)。

5.3 glibc 版本不兼容

现象: `GLIBC_2.34 not found` 错误。

根因: 宿主机 Ubuntu 22.04 (glibc 2.35) 编译的二进制无法在 Debian 11 VM (glibc 2.31) 运行。

解决方案: 所有 PoC 必须加 `-static` 标志编译。

5.4 AF_VSOCK 设备不可用

现象: `No such device` 错误。

根因: QEMU 未提供 `virtio-vsock-pci` 设备。

解决方案: 在 QEMU 启动参数中添加 `-device vhost-vsock-pci,guest-cid=3`。

5.5 KernJC 自动化环境的局限性

1. **内核配置不完整:** KernJC 基于 Kconfig 图的配置解析未自动识别 VSOCK 和 `userfaultfd` 依赖, 需要手动干预。
2. **QEMU 参数硬编码:** 默认 CPU 模型 (`host`) 在无 KVM 环境下导致启动失败; 缺少 `virtio-vsock` 等外设支持。
3. **模块管理缺失:** `rootfs` 镜像不包含 `kmod`, 导致依赖内核模块 (`.ko` 文件) 的 PoC 无法正常运行。
4. **glibc 兼容性:** `rootfs` 和宿主环境的 glibc 版本差异未处理。

6 结论

本实验成功复现了 CVE-2021-26708 漏洞, 获得了完整的 KASAN use-after-free 报告, 验证了 VSOCK 竞争条件漏洞在 5.10.12 内核上的存在。实验过程中识别并克服了 KernJC 工具在配置解析、模块管理、设备模拟和库兼容性方面的多项工程限制。

对于未来的自动化漏洞复现工作, 建议在 KernJC 基础上增加: (1) 更全面的 Kconfig 依赖图解析算法; (2) QEMU 设备的自动探测与适配; (3) `rootfs` 镜像的模块管理工具预安装。

参考文献

- [1] Bonan Ruan, Jiahao Liu, Chuqi Zhang, Zhenkai Liang. *KernJC: Automated Vulnerable Environment Generation for Linux Kernel Vulnerabilities*. In Proceedings of RAID '24, Padua, Italy, 2024.